

marks it as an optional field that can be left blank and can be stored as null in the database

- ◆ A publish status field that can be toggled between published / unpublished, it default to False, or not published

Note that we have used an 'ImageField' in our model, and that particular feature of Django requires that we install a Python library called Pillow. So go ahead and run 'pip install Pillow'. On Linux systems this might require you to have a development environment set up. If pip fails to install Pillow, check your distro's repositories for a version of Pillow for Python 3 and install from there.

You will also notice a '`_str_`' method that simply returns the title of the article. If we omit this then Django does not know how to give this model a name, and thus by default uses names that are unhelpful. Here we are telling it that the title of the article is what this piece of content should be called.

The Meta class inside the model class sets up two properties. The first tells Django that the plural of Article is Articles. In this case it is entirely unnecessary since Django is quite capable of doing this itself, it can even handle pluralising some common words where the plural isn't simply the singular with an 's' in the end. You need only do this if you notice that Django has got the plural wrong. The second property in the Meta class tells Django that this model is to be ordered based on the creation date of the article.

One of the hallmarks of relational databases, are well, relations. We have seen a bit of that above with the ForeignKey field that links the Article to a User, but relations between different tables are something that we should probably discuss in a little more detail.

Relations Between Models

The way relational databases, like the ones that use SQL store complex hierarchical data is through the use of relations. One piece of data in one table can have further pieces of data in other tables associated with it. The way these are tracked is by giving each piece of data a unique ID that can be referenced by other pieces of data. Let's look at this through a few examples.

Imagine we want to store information of different companies and their employees. Now each company can have multiple employees, but each employee can only be part of one company. This kind of relation going from a company to its employees is called a One to Many relationship, since we have one company associated with many employees. Going from